

Implementation of JPEG Compression on a 6×6 Image Block

1. Problem Statement

The objective is to compress a 6×6 grayscale image block using JPEG compression techniques. The compression involves transforming the spatial image data into the frequency domain, reducing less important information, and encoding the data efficiently to produce a compressed bitstream.

The specific goals are:

1. Reducing the data size while retaining important visual information.
2. Applying lossy compression (DCT + Quantization) to remove less perceptible details.
3. Applying lossless compression (RLE + Huffman coding) to efficiently encode the data.

2. Steps of JPEG Compression

Step 1: Level Shifting

- Subtracting 128 from each pixel to center values around zero.
- Formula: $f(x,y)=I(x,y)-128$

Step 2: Discrete Cosine Transform (DCT)

- Transforming the 2D image block into frequency domain.
- Formula:

$$F(u,v)=\frac{2}{N} C(u)C(v)\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(x,y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

- $C(k)=\frac{1}{\sqrt{2}}$ for $k=0$, else 1

Step 3: Quantization

- Divide DCT coefficients by a quantization matrix and round to reduce less important details.
- Formula: $F_q(u,v)=\text{round}(F(u,v)/Q(u,v))$

Step 4: Zigzag Scanning

- Reorder the 2D quantized coefficients into a 1D array.
- Purpose: Low-frequency values come first, high-frequency zeros cluster at the end.

Step 5: Run-Length Encoding (RLE)

- Represent consecutive zeros efficiently as (count, value) pairs.
- Append End of Block (EOB) symbol at the end.

Step 6: Huffman Encoding

- Count the frequency of each RLE symbol.
- Build a Huffman tree to assign variable-length codes.
- Replace each RLE symbol with its corresponding Huffman code to generate the final bitstream.

3. Python Code:

```
import numpy as np
import math
from collections import Counter
import heapq

I = np.array([
    [52, 55, 61, 66, 70, 61],
    [63, 59, 55, 90, 109, 85],
    [62, 59, 68, 113, 144, 104],
    [63, 58, 71, 122, 154, 106],
    [67, 61, 68, 104, 126, 88],
    [68, 65, 60, 70, 77, 68]
], dtype=float)

N = 6

f = I - 128

def C(k):
    return 1 / math.sqrt(2) if k == 0 else 1

F = np.zeros((N, N))

for u in range(N):
    for v in range(N):
        sum_val = 0
        for x in range(N):
            for y in range(N):
                sum_val += (
                    f[x][y]
                    * math.cos((2*x+1)*u*math.pi/(2*N))
                    * math.cos((2*y+1)*v*math.pi/(2*N))
                )
```

```

        F[u][v] = (2/N) * C(u) * C(v) * sum_val

Q = np.array([
    [16, 11, 10, 16, 24, 40],
    [12, 12, 14, 19, 26, 58],
    [14, 13, 16, 24, 40, 57],
    [14, 17, 22, 29, 51, 87],
    [18, 22, 37, 56, 68, 109],
    [24, 35, 55, 64, 81, 104]
])

Fq = np.round(F / Q)

def zigzag(matrix):
    rows, cols = matrix.shape
    result = []
    for s in range(rows + cols - 1):
        if s % 2 == 0:
            for i in range(s, -1, -1):
                j = s - i
                if i < rows and j < cols:
                    result.append(int(matrix[i][j]))
        else:
            for j in range(s, -1, -1):
                i = s - j
                if i < rows and j < cols:
                    result.append(int(matrix[i][j]))
    return result

Z = zigzag(Fq)

def rle(arr):
    result = []
    count = 0
    for val in arr:
        if val == 0:
            count += 1
        else:
            result.append((count, val))
            count = 0
    result.append(("EOB", 0))
    return result

rle_data = rle(Z)

```

```

flat_symbols = [str(item) for item in rle_data]
freq = Counter(flat_symbols)

heap = [[weight, [symbol, ""]] for symbol, weight in freq.items()]
heapq.heapify(heap)

while len(heap) > 1:
    lo = heapq.heappop(heap)
    hi = heapq.heappop(heap)

    for pair in lo[1:]:
        pair[1] = '0' + pair[1]
    for pair in hi[1:]:
        pair[1] = '1' + pair[1]

    heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])

huffman_codes = sorted(heapq.heappop(heap)[1:], key=lambda p: (len(p[-1]), p))

huffman_dict = {symbol: code for symbol, code in huffman_codes}

encoded_output = ""
for item in flat_symbols:
    encoded_output += huffman_dict[item]

print("Zigzag:\n", Z)
print("\nRLE:\n", rle_data)
print("\nHuffman Codes:")
for k, v in huffman_dict.items():
    print(k, ":", v)

print("\nFinal Encoded Bitstream:\n", encoded_output)

```